

Manual Técnico

El Impostor



2 DAM
Programación Móvil
Sara Thapa Kc

ÍNDICE DE CONTENIDOS

Introducción

- 1. Arranque y Navegación (splash_screens.dart y home_screen.dart)**
- 2. Login (login_screen.dart)**
- 3. Registro (registro.dart)**
- 4. Fichero de variables Globales (globals.dart)**
- 5. Preparación y Persistencia Local (agregar_jugadores.dart)**
- 6. Motor del Juego y Consumo de API (jugar.dart)**
- 7. Componentes Custom y Animaciones (flipCard.dart)**
- 8. Resolución (final.dart)**

Introducción

El juego se llama El Impostor, está basado en un juego que se popularizó en Tik Tok. Trata sobre cómo hay 1 impostor en la partida, es decir, todo el mundo obtiene la misma palabra secreta excepto una persona. Los demás tienen que intentar adivinar quién es el impostor diciendo por turnos una palabra relacionada con la palabra secreta. Para más información consultar el [Manual de Usuario](#).

El proyecto está hecho con Flutter en el lenguaje de Dart. Se ha logrado su funcionalidad sobre todo gracias al fichero globals.dart el cual contiene variables globales y ha facilitado todo.

Todo el sistema de usuarios tira de Firebase Authentication, separado en dos vistas para mantener el código limpio.

1. Arranque y Navegación (splash_screens.dart y home_screen.dart)

El inicio de la app.

- ❖ **SplashScreen**: He usado el paquete “animated_splash_screen”. Carga un GIF y, tras 1100 milisegundos de espera, dirige directamente hacia la pantalla principal (HomeScreen).
- ❖ **HomeScreen**: Es un Scaffold contenedor muy básico que tiene un color de fondo gris oscuro y carga directamente el widget del login (LoginScreen).

2. Login (login_screen.dart)

- ❖ He montado un formulario clásico controlando el estado de los inputs con `_correo` y `_password` mediante el evento `onChanged` de los `TextField`.
- ❖ La función `_loginUser()` es la base aquí. Antes de lanzar la petición asíncrona a Firebase (`signInWithEmailAndPassword`), valido que los campos no estén vacíos para poder proseguir.
- ❖ Todo el bloque está envuelto en un `try-catch`. Se controlan excepciones concretas de `FirebaseAuthException` (cómo `user-not-found` o `wrong-password`) para pintar en pantalla un mensaje de error que el usuario entienda, en lugar de un pantallazo rojo.
- ❖ Si el login va bien, uso `Navigator.pushReplacement` hacia la pantalla de Jugadores. Esto es importante para destruir la ruta de login; así, si el usuario le da al botón físico de "atrás" en su móvil, no vuelve al formulario de inicio de sesión.

3. Registro (registro.dart)

- ❖ Sigue un patrón idéntico al login, pero utilizando el método `createUserWithEmailAndPassword`.
- ❖ Captura errores típicos de la creación de cuentas, como `weak-password` (cuando Firebase detecta que la contraseña es muy corta) o `email-already-in-use`. Tras registrarse, devuelve al usuario al login.
- ❖ También contamos con un botón para volver hacia atrás (al login) en caso de que no se necesite registrar y haya sido un error.

4. Fichero de variables Globales (globals.dart)

Este fichero es básicamente la memoria central del juego. Lo uso para no tener que estar pasando datos por constructor de pantalla en pantalla ya que es mucho más sencillo crear un fichero global ya que es lo más óptimo.

- ❖ **jugadores (int)**: Empieza en 3. Es el contador que define el tamaño del grid y cuántas tarjetas se van a generar.
- ❖ **nombresJugadores (List<String>)**: Un array dinámico que guarda los nombres de los jugadores.
- ❖ **numeroAleatorio (late int)**: La declaro con late porque no le doy valor hasta que le damos al botón de jugar. Básicamente guardará el índice del array que corresponde al jugador que se lleva el rol de impostor.

5. Preparación y Persistencia Local (agregar_jugadores.dart)

Es el fichero para realizar el aumento o el decremento de jugadores. Aquí se especifican muchos datos que nos sirven para más adelante.

- ❖ **Persistencia (SharedPreferences)**: Para que los usuarios no tengan que escribir los nombres cada vez que abren la app, uso SharedPreferences.
 - En el initState(), ejecuto cargarNombres(), que lee las claves del tipo jugador_0, jugador_1, etc. Si no encuentra nada (usando el operador ??), pone un nombre genérico.
 - Cada vez que se modifica un TextFormField, se lanza guardarNombres() para actualizar la memoria del dispositivo.
- ❖ **Lógica de botones**: La función eliminarJugadores() tiene un funcionamiento sencillo: un if (jugadores > 3) para evitar que la partida se rompa, ya que el juego no tiene sentido con menos de tres personas.
- ❖ **Validación en caliente**: Al darle al botón de jugar, recorro el array de nombres con un bucle for. Si algún usuario ha borrado su nombre (comprobado con .trim().isEmpty), el código le añade un nombre por defecto ("Jugador X") para que en la siguiente pantalla no se muestre con tarjetas en blanco.

6. Motor del Juego y Consumo de API (jugar.dart)

Este fichero es el que contiene la lógica para poder jugar

- ❖ **Selección del Impostor:** En el initState(), uso Random().nextInt(jugadores) de la librería dart:math para generar un número aleatorio basado en la cantidad de participantes. Ese número se lo enchufo a la variable global numeroAleatorio.
- ❖ **Petición HTTP:** La función palabraAleatoria(), mediante un método GET, llama a una API pública (random-words-api) usando el paquete http.
 - Si el statusCode es 200, parseo el JSON de respuesta con jsonDecode y me guardo el valor de la clave word.
 - *Seguridad:* Esto es importante. Toda la llamada de red está dentro de un try-catch. Si la API se cae, responde mal, o el móvil no tiene cobertura, capto el error y asigno a la variable la palabra "Nutria". Así garantizo que la app no se vuelva inutilizable y puedan seguir jugando.
- ❖ La vista se monta con un GridView.count que va instanciando componentes FlipCardWidget. Le paso como parámetro un booleano (diferente) que evalúa si el índice de ese bucle coincide con el numeroAleatorio, si es verdadero, a ese jugador le toca ser el impostor.

7. Componentes Custom y Animaciones (flipCard.dart)

El componente visual más técnico del proyecto. Emula el giro físico de una tarjeta. Sacado de un tutorial de Youtube:

📺 Master Flip Card Animations in Flutter: A Step-by-Step Tutorial for...

- ❖ Se ha implementado el mixin SingleTickerProviderStateMixin para poder enlazar el controlador de la animación a los frames de la pantalla.
- ❖ El AnimationController gestiona la duración (300 ms) y usa un Tween con una curva CurvedAnimation para suavizar el inicio y el fin del giro.
- ❖ Como al girar en 3D el texto de atrás se renderizaría al revés, lo envuelvo en otro Transform.scale(scaleX: -1) para voltearlo en el eje X y que sea legible. Dependiendo del flag diferente, esa parte trasera pintará el layout del impostor o el layout de la palabra.

8. Resolución (final.dart)

Esta es la pantalla final de la aplicación.

- ❖ Al inicio, declaro la variable impostor consultando directamente la posición del array global: `nombresJugadores[numeroAleatorio]`.
- ❖ Se muestra el resultado en pantalla y dejo un `ElevatedButton` que hace un `Navigator.push` de vuelta a la clase `Jugadores`. Al no resetear las variables globales, se conservan los nombres para empezar la siguiente ronda fácilmente.